PARTE 1

RottenPotatoes com Rails e RubyMine

Guia prático para desenvolvimento acompanhado em sala de aula

PROJETO	Catálogo de filmes com CRUD, validações, ordenação e testes
TECNOLOGIAS	Ruby, Rails, SQLite, Haml, Minitest e RubyMine
ESCOPO	Da criação do projeto até testes e depuração. Git será tratado na Parte 2.

Resultado esperado

Ao final desta parte, cada aluno terá uma aplicação Rails funcional para cadastrar, listar, visualizar, editar, excluir e ordenar filmes, com regras de negócio testadas.

ETAPA 0

Visão geral da aula

Este material foi planejado para uma prática guiada. Em cada etapa, o professor demonstra, os alunos reproduzem, executam a aplicação e validam o resultado.

Objetivos de aprendizagem

- Reconhecer a estrutura de uma aplicação Rails e o papel do padrão MVC.
- Criar recursos com generators e compreender os arquivos produzidos.
- Usar migrations, Active Record e o console Rails.
- Escrever views em Haml e compreender sua sintaxe baseada em indentação.
- Implementar validações e melhorar formulários Rails.
- Receber parâmetros, ordenar consultas e proteger os campos aceitos.
- Executar testes Minitest e acompanhar o código com o depurador do RubyMine.

Ciclo didático recomendado

Ação no RubyMine	O que observar
1. Apresentar o comportamento	Começar pelo que o usuário deverá conseguir fazer.
2. Demonstrar no RubyMine	Executar uma ação curta e narrar o caminho percorrido.
3. Mostrar o comando equivalente	Relacionar a interface da IDE às ferramentas do Rails.
4. Alunos reproduzem	Circular pela sala e observar dificuldades recorrentes.
5. Verificar	Abrir no navegador, observar a saída e executar testes.

Nota de escopo: não faremos commits ou operações de Git nesta parte. Esse conteúdo será introduzido na próxima semana.

ETAPA 1

Preparar o ambiente

Padronize as versões utilizadas pela turma. Diferenças de Ruby ou Rails podem produzir arquivos, dependências e comportamentos distintos.

Pré-requisitos

- Ruby e Rails
- RubyMine
- SQLite
- Git instalado apenas como pré-requisito do ambiente
- No Windows: ambiente Rails via WSL

Verificação no terminal integrado

```
ruby --version
rails --version
git --version
```

Checklist antes de iniciar

[ ] O RubyMine reconhece um interpretador Ruby.

[ ] A versão do Rails adotada pela turma está disponível.

[ ] O terminal integrado executa Ruby e Rails.

[ ] A máquina consegue abrir localhost no navegador.

Atenção do professor

Se o RubyMine não encontrar o Ruby, corrija primeiro o campo Interpreter. Instalar novamente o Rails sem resolver o interpretador costuma apenas esconder o problema.

ETAPA 2

Criar o projeto e configurar Haml

Na tela inicial do RubyMine, selecione New Project e depois Rails > Application.

Ação no RubyMine	O que observar
Name: rottenpotatoes	O nome também será usado na pasta do projeto.
Interpreter	Selecione o Ruby padronizado para a turma.
Rails version	Escolha a mesma versão em todas as máquinas.
Database: SQLite	É suficiente para o desenvolvimento local desta parte.
Create	Aguarde a instalação das dependências finalizar.

Comando equivalente

```
rails new rottenpotatoes
```

Adicionar Haml antes de gerar as views

Abra o Gemfile e acrescente a gem que integra os generators do Rails ao Haml:

```
gem "haml-rails"
```

Salve o arquivo e instale as dependências em Tools > Bundler > Install ou no terminal integrado:

```
bundle install
```

Execute a configuração Rails pelo botão verde Run e abra:

```
http://localhost:3000
```

Conhecendo o projeto

```
app/
controllers/
models/
views/
config/routes.rb
db/migrate/
db/seeds.rb
test/
```

Haml usa a extensão .html.haml e a indentação define a estrutura. Espaços inconsistentes podem alterar ou invalidar a view.

ETAPA 3

Gerar o recurso Movie

Antes de executar o generator, apresente o modelo de dados.

Ação no RubyMine	O que observar
title : string	Título curto do filme.
rating : string	Classificação: G, PG, PG-13, R ou NC-17.
description : text	Sinopse, potencialmente mais longa.
release_date : date	Data de lançamento sem horário.

No RubyMine

Abra Tools > Run Rails Generator, selecione scaffold e informe os argumentos abaixo. A opção --skip-routes impede que o generator edite routes.rb automaticamente:

```
Movie title:string rating:string description:text release_date:date --skip-routes
```

Comando equivalente

```
bin/rails generate scaffold Movie title:string rating:string description:text release_date:date
--skip-routes
```

O que o scaffold cria

- migration e model Movie
- MoviesController
- views .html.haml para o CRUD
- arquivos de teste
- nenhuma rota de movies, pois foi usada --skip-routes

Confirme na saída do generator que as views têm extensão .html.haml e que não apareceu a linha route resources :movies.

ETAPA 4

Declarar o recurso, migrar e compreender o MVC

Abra config/routes.rb. Dentro do bloco Rails.application.routes.draw, os alunos devem escrever manualmente:

```
Rails.application.routes.draw do
resources :movies
end
```

Uma declaração resources cria as rotas REST de index, show, new, create, edit, update e destroy. Verifique o resultado:

```
bin/rails routes
```

Peça aos alunos que encontrem os verbos GET, POST, PATCH/PUT e DELETE e os relacionem às ações do MoviesController.

Depois, pressione Ctrl duas vezes, procure db:migrate e execute a tarefa. Também é possível usar o terminal integrado:

```
bin/rails db:migrate
```

Abra db/schema.rb e localize a tabela movies. Em seguida, visite:

```
http://localhost:3000/movies
```

Caminho de uma requisição

GET /movies	rota	index	Movie.all	index.html.haml

Arquivos para abrir lado a lado

```
config/routes.rb
app/models/movie.rb
app/controllers/movies_controller.rb
app/views/movies/index.html.haml
```

Atividade rápida

[ ] Criar um filme pelo formulário.

[ ] Abrir a página de detalhes.

[ ] Editar o título.

[ ] Excluir o registro.

ETAPA 5

Experimentar Active Record e validações

Abra Tools > Run Rails Console. Use o console para observar objetos e consultas antes de alterar o controller.

```
Movie.create!(
title: "Toy Story",
rating: "G",
description: "Brinquedos ganham vida.",
release_date: Date.new(1995, 11, 22)
)

Movie.count
Movie.all
Movie.where(rating: "G")
Movie.order(:title)
```

Abra app/models/movie.rb e adicione:

```
class Movie < ApplicationRecord
RATINGS = %w[G PG PG-13 R NC-17].freeze

validates :title, presence: true
validates :rating, inclusion: { in: RATINGS }
validates :release_date, presence: true
end
```

Teste as regras no console

```
movie = Movie.new
movie.valid?
movie.errors.full_messages
```

Ideia central: o formulário orienta a entrada, mas é o model que protege a regra de negócio.

ETAPA 6

Editar views Haml e criar dados iniciais

Antes de editar o formulário, compare uma pequena estrutura HTML com Haml:

```
HTML Haml
<h1>Filmes</h1> %h1 Filmes
<p class="notice">Oi</p> %p.notice Oi
```

Em Haml, = imprime o resultado de uma expressão Ruby; - executa Ruby sem imprimir. A indentação substitui as tags de fechamento.

Abra app/views/movies/_form.html.haml e substitua o campo livre de rating por uma seleção:

```
.field
= form.label :rating
= form.select :rating, Movie::RATINGS
```

Em db/seeds.rb, adicione um conjunto pequeno e variado de filmes:

```
movies = [
{ title: "Toy Story", rating: "G",
release_date: Date.new(1995, 11, 22),
description: "Brinquedos ganham vida." },
{ title: "The Matrix", rating: "R",
release_date: Date.new(1999, 3, 31),
description: "Um programador descobre a realidade." },
{ title: "Star Wars", rating: "PG",
release_date: Date.new(1977, 5, 25),
description: "Uma aventura em uma galáxia distante." }
]

movies.each do |attributes|
Movie.find_or_create_by!(title: attributes[:title]) do |movie|
movie.assign_attributes(attributes)
end
end
```

Execute pelo terminal integrado ou pela tarefa db:seed:

```
bin/rails db:seed
```

Verificação

[ ] A listagem apresenta três filmes.

[ ] Executar db:seed novamente não duplica os filmes.

ETAPA 7

Implementar ordenação segura

Comportamento esperado: o usuário clica em Título ou Lançamento e a URL registra o critério escolhido.

No método index de MoviesController

```
def index
allowed_sorts = %w[title release_date]
@sort = params[:sort] if allowed_sorts.include?(params[:sort])
@movies = @sort ? Movie.order(@sort) : Movie.all
end
```

Na tabela de app/views/movies/index.html.haml

```
%th= link_to "Título", movies_path(sort: "title")
%th Classificação
%th= link_to "Lançamento", movies_path(sort: "release_date")
```

Por que allowed_sorts?

O parâmetro vem do navegador. A lista explícita limita a ordenação aos campos previstos pela aplicação.

Teste manual

[ ] /movies?sort=title coloca os títulos em ordem alfabética.

[ ] /movies?sort=release_date ordena pelas datas.

[ ] /movies?sort=qualquer não provoca erro.

ETAPA 8

Depurar e testar

Coloque um breakpoint na primeira linha do método index. Execute o servidor com Debug e visite /movies?sort=title.

Ação no RubyMine	O que observar
Inspecionar params	Confirmar que sort contém title.
Avançar linha por linha	Observar a validação contra allowed_sorts.
Inspecionar @movies	Identificar a relação Active Record antes da renderização.

Teste do model

```
require "test_helper"

class MovieTest < ActiveSupport::TestCase
test "precisa ter título" do
movie = Movie.new(rating: "PG", release_date: Date.current)

assert_not movie.valid?
assert_includes movie.errors[:title], "can't be blank"
end

test "aceita classificações conhecidas" do
movie = Movie.new(title: "Filme", rating: "INVALIDA",
release_date: Date.current)

assert_not movie.valid?
end
end
```

Clique no triângulo verde ao lado do teste, da classe ou da pasta test. O RubyMine exibirá os resultados na janela Run.

Ciclo de aprendizagem

teste falha > implementar > teste passa > refatorar

ETAPA 9

Fechamento da primeira parte

Demonstração final esperada

[ ] Abrir a listagem de filmes.

[ ] Cadastrar um filme válido.

[ ] Tentar cadastrar um filme inválido e observar as mensagens.

[ ] Editar e excluir um filme.

[ ] Ordenar por título e por data.

[ ] Executar todos os testes com sucesso.

[ ] Usar um breakpoint para observar params.

Perguntas para revisão

- Qual é a diferença entre uma migration e um model?
- Qual componente recebe os parâmetros da requisição?
- Por que a validação deve permanecer no model mesmo com um campo select?
- O que o scaffold criou e por que a rota não foi gerada automaticamente?
- O que resources :movies acrescentou à aplicação?
- Qual é a diferença entre = e - em uma view Haml?
- Por que limitamos os campos aceitos para ordenação?

Próxima semana - Parte 2

Controle de versão com Git integrado ao RubyMine, histórico de alterações, branches e continuidade das funcionalidades do RottenPotatoes.

Referências

Ruby on Rails Guides: https://guides.rubyonrails.org/getting_started.html

JetBrains RubyMine Help: https://www.jetbrains.com/help/ruby/create-and-run-your-first-project.html

RubyMine - Rails generators: https://www.jetbrains.com/help/ruby/creating-rails-application-elements.html

RubyMine - Running tests: https://www.jetbrains.com/help/ruby/performing-tests.html

Rails Routing: https://guides.rubyonrails.org/routing.html

Haml: https://github.com/haml/haml